

Scripts de déploiement

VPN WireGuard & Stack Docker Compose

Réalisation 02 — Nomad-Security

Serveur	srv-gw-sec (192.168.20.30) — Ubuntu Server
Rôle	Passerelle VPN + Reverse Proxy + MFA
Composants	WireGuard, Docker, Nginx Proxy Manager, Authelia
Réseau VPN	10.0.0.0/24

1. Configuration WireGuard

WireGuard assure le tunnel chiffré entre les collaborateurs nomades et le réseau interne de l'entreprise. La configuration ci-dessous active le routage NAT (MASQUERADE) permettant aux clients VPN d'accéder au LAN (192.168.20.0/24).

1.1. Configuration serveur

Fichier : **wg0.conf** (/etc/wireguard/ — srv-gw-sec)

```
[Interface]
PrivateKey = 2BHztptn06p0F89H/w88HEQg7keiEMyZ3fPM1d/1MB1k=
Address = 10.0.0.1/24
ListenPort = 51820
PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -A FORWARD -o %i -j ACCEPT; iptables -t
nat -A POSTROUTING -o ens18 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D FORWARD -o %i -j ACCEPT; iptables
-t nat -D POSTROUTING -o ens18 -j MASQUERADE

[Peer]
PublicKey = TG9QYUZVQYSgA8L5a4t5WQyzBQaA2ip0SFGaReIfVjM=
AllowedIPs = 10.0.0.2/32
```

- PostUp / PostDown : règles iptables activant le forward et le NAT vers l'interface LAN (ens18).
- AllowedIPs côté serveur : restreint le pair client à sa seule IP VPN (10.0.0.2/32) pour limiter la surface d'attaque.

Point de vigilance constaté en exploitation : la règle PostDown doit utiliser l'option -D (delete) sur chacune des trois commandes iptables, symétriquement à PostUp (-A / append). Une coquille laissant un -A sur la règle NAT POSTROUTING dans PostDown empile une règle MASQUERADE supplémentaire à chaque redémarrage du tunnel (wg-quick down/up) sans jamais la retirer, ce qui finit par dupliquer le NAT et complique le diagnostic réseau.

1.2. Configuration client (exemple Windows)

Fichier : NOMAD-VPN.conf (*Application WireGuard — poste client*)

```
[Interface]
PrivateKey = <clé privée générée par le client>
Address = 10.0.0.2/24
DNS = 192.168.20.10

[Peer]
PublicKey = TG9QYUZVQYSgA8L5a4t5WQyzBQaA2ip0SFGaReIfVjM=
AllowedIPs = 10.0.0.0/24, 192.168.20.0/24
Endpoint = 172.16.1.33:51820
PersistentKeepalive = 25
```

- AllowedIPs côté client : inclut le réseau VPN ET le réseau LAN (192.168.20.0/24) pour permettre l'accès aux services internes (AD, GLPI).
- DNS : pointe vers le contrôleur de domaine (192.168.20.10) pour la résolution des noms internes (*.resilience.local).
- PersistentKeepalive : maintient le tunnel actif derrière un NAT en envoyant un paquet toutes les 25 secondes.

2. Stack Docker Compose

La stack applicative (Reverse Proxy + MFA) est déployée par conteneurs pour isoler les services et faciliter leur mise à jour.

Fichier : docker-compose.yml (*/opt/nomad/ — srv-gw-sec*)

```
version: '3'
services:
  nginx-proxy-manager:
    image: jc21/nginx-proxy-manager:latest
    ports:
      - "80:80"
      - "443:443"
      - "81:81"
    volumes:
      - ./npm/data:/data
      - ./npm/letsencrypt:/etc/letsencrypt
    restart: unless-stopped

  authelia:
    image: authelia/authelia:latest
    volumes:
      - ./authelia:/config
    ports:
      - "9091:9091"
    restart: unless-stopped
    healthcheck:
      disable: true
```

- nginx-proxy-manager : reverse proxy gérant les certificats SSL et la publication des applications internes.
- authelia : portail d'authentification centralisée assurant le MFA, lié à l'Active Directory via LDAP.
- healthcheck désactivé sur Authelia : le test de santé intégré à l'image utilise du HTTP en clair, incompatible avec l'écoute TLS configurée sur le port 9091.

3. Configuration Authelia

Ce fichier définit la liaison LDAP vers l'Active Directory, les règles d'accès par domaine et la politique de seconde authentification.

Fichier : configuration.yml (*/opt/nomad/authelia/ — srv-gw-sec*)

```
server:
  address: tcp://0.0.0.0:9091
  timeouts:
    write: 30s
  tls:
    certificate: '/config/cert.pem'
    key: '/config/key.pem'

authentication_backend:
  ldap:
    address: ldap://192.168.20.10
    base_dn: DC=resilience,DC=local
    attributes:
      username: sAMAccountName
    additional_users_dn: OU=Utilisateurs
    users_filter: (&({username_attribute}={input})(objectClass=user))
    groups_filter: (member={dn})
    user: CN=svc-authelia,OU=SERVICES,DC=resilience,DC=local
    password: 'Resilience2026'

session:
  secret: nomad_secret_2026
  expiration: 1h
  inactivity: 15m
  cookies:
    - domain: resilience.local
      authelia_url: https://auth.resilience.local
      default_redirection_url: https://glpi.resilience.local

storage:
  encryption_key: un_encryption_key_2026
  local:
    path: /config/db.sqlite3

notifier:
  filesystem:
    filename: /config/notification.txt

access_control:
  default_policy: 'deny'
  rules:
    - domain: 'auth.resilience.local'
```

```

    policy: 'bypass'
  - domain: 'glpi.resilience.local'
    policy: 'two_factor'

totp:
  issuer: 'Resilience-IT'
  period: 30
  skew: 1

```

- `authentication_backend.ldap` : connexion au contrôleur de domaine via le compte de service `svc-authelia`.
- `access_control` : politique par défaut «deny» ; seul `glpi.resilience.local` impose le second facteur (`two_factor`).
- `server.tls` : active l'écoute chiffrée sur le port 9091, requise par les versions récentes d'Authelia pour la transmission sécurisée des cookies de session.
- `session.expiration` / `session.inactivity` : sans ces valeurs explicites, Authelia applique une durée d'inactivité par défaut très courte ; la session est alors jugée expirée entre deux requêtes, ce qui invalide le cookie côté `/api/verify` même juste après une authentification réussie.

4. Configuration Nginx — Intégration Authelia / GLPI

Cette configuration personnalisée, ajoutée dans l'onglet Advanced du Proxy Host `glpi.resilience.local`, déclenche la vérification Authelia avant de transmettre la requête à GLPI, puis transmet l'identité de l'utilisateur via des en-têtes HTTP (SSO).

Fichier : `server_proxy.conf` (extrait) *(Nginx Proxy Manager — onglet Advanced)*

```

# Configuration Nginx personnalisée - Proxy Host glpi.resilience.local
# Onglet "Advanced" dans Nginx Proxy Manager

location / {
    # 1. Verification d'authentification auprès d'Authelia
    auth_request /authelia/api/verify;

    # 2. Recupere l'identite renvoyee par Authelia et la transmet à GLPI
    auth_request_set $user $upstream_http_remote_user;
    auth_request_set $groups $upstream_http_remote_groups;
    auth_request_set $name $upstream_http_remote_name;
    auth_request_set $email $upstream_http_remote_email;
    proxy_set_header Remote-User $user;
    proxy_set_header Remote-Groups $groups;
    proxy_set_header Remote-Name $name;
    proxy_set_header Remote-Email $email;

    # 3. Session expiree ou absente (401) : redirection vers le portail MFA
    error_page 401 =302 https://auth.resilience.local/?rd=$scheme://$http_host$request_uri;

    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $http_connection;
    proxy_http_version 1.1;
    include conf.d/include/proxy.conf;
}

```

```

}

# 4. Sous-requete interne de verification aupres d'Authelia
location /authelia/api/verify {
    internal;
    proxy_pass https://192.168.20.30:9091/api/verify;
    proxy_ssl_verify off;

    # Host DOIT inclure le port :9091 : Authelia valide la session
    # en comparant ce Host à son authelia_url configure ; sans le port,
    # la session est rejetee meme avec un cookie valide.
    proxy_set_header Host auth.resilience.local:9091;
    proxy_set_header Cookie $http_cookie;

    proxy_set_header X-Original-URL $scheme://$http_host$request_uri;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-Host $http_host;
    proxy_set_header X-Forwarded-Uri $request_uri;

    # Indispensable pour les requetes POST (creation de ticket, ajax) :
    # sans ces deux lignes, auth_request tente de relayer le corps de
    # la requête d'origine vers Authelia et provoque un timeout (408),
    # qui se traduit par une erreur 500 cote GLPI.
    proxy_set_header Content-Length "";
    proxy_pass_request_body off;
}

```

- `location /` vs `server { }` : les directives `auth_request`, `auth_request_set` et `proxy_set_header Remote-*` doivent être déclarées DANS le bloc `location /` et non au niveau `server`. Nginx Proxy Manager génère son propre bloc `location /` ; coller ces directives en dehors (dans l'onglet Advanced classique, hors `location`) les rend inopérantes pour les requêtes qui matchent `location /` — l'en-tête `Remote-User` n'arrive alors jamais jusqu'à GLPI, sans aucune erreur visible.
- `Remote-User` / `Remote-Groups` / `Remote-Name` / `Remote-Email` : en-têtes transmis à GLPI pour l'authentification automatique (`REMOTE_USER`), évitant une double saisie du mot de passe.
- `Host auth.resilience.local:9091` (avec le port) : Authelia valide le cookie de session en comparant le Host reçu à son `authelia_url` configuré. Un Host sans port (`auth.resilience.local`) fait échouer cette correspondance et la session, bien que valide, est rejetée comme anonyme — sans message d'erreur explicite côté Nginx.
- `proxy_pass_request_body off` / `Content-Length ""` : sans ces deux lignes, `auth_request` relaie le corps des requêtes POST (création de ticket, appels ajax GLPI) vers Authelia, ce qui provoque un timeout (408) côté Authelia et se traduit par une erreur 500 dans GLPI lors de la soumission de formulaires.
- `error_page 401` : redirige automatiquement vers le portail Authelia avec l'URL de retour (paramètre `rd`) en cas de session expirée.
- Côté GLPI (Configuration > Authentification > Autres méthodes d'authentification) : le champ «Champ de stockage de l'identifiant dans la requête HTTP» doit être réglé sur `REMOTE_USER` (GLPI lit alors `$_SERVER['HTTP_REMOTE_USER']`). Une règle dans Administration > Règles > Règles d'attribution des autorisations à un utilisateur, avec le critère Type d'authentification = Authentifications externes, est obligatoire pour autoriser ces connexions ; sans elle, GLPI refuse l'accès même avec un en-tête valide.